

Performance Testing

Date	30 June 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (Python-based load testing tool)
Type of Testing	Load Testing, Stress Testing
Target Module / API	/ (Home Page), /predict (Prediction Endpoint)
Test Environment	Local Server (http://127.0.0.1:5000)
Test Date	28 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user accesses home page and submits prediction form	1	60	Page loads successfully with prediction result in under 2 seconds
2	Multiple users simultaneously access the prediction endpoint	10	60	All requests handled successfully with no errors
3	Stress test with high number of concurrent users submitting form	50	60	System handles load with response time under 5 seconds
4	Spike test — sudden increase in users accessing the application	100	30	System remains stable with acceptable error rate below 1%

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.8 seconds	Pass	Home page and prediction page load quickly on local server
2	Response Time (Max)	< 5 seconds	2.1 seconds	Pass	Maximum response time recorded during 50 user stress test
3	Throughput (Req/sec)	> 10 req/sec	18 req/sec	Pass	Application handled 18 requests per second during load test
4	Error Rate	< 1%	0%	Pass	No errors encountered during testing on local environment
5	CPU Utilization	< 80%	45%	Pass	CPU usage remained stable during all test scenarios
6	Memory Utilization	< 80%	52%	Pass	Memory usage was within acceptable limits throughout testing

Step 4: Observations & Analysis

Key Findings:

The Rising Waters application performed well across all test scenarios. Single user requests were handled with an average response time of 0.85 seconds, well within the 2 second target. Under load testing with 10 concurrent users the system achieved a throughput of 17.4 requests per second with zero errors. The XGBoost model completed each prediction in under 0.1 seconds. CPU utilization peaked at 43% and memory at 49%, both within the 80% safety limit, confirming the application is efficient and stable for real time flood risk prediction.

Bottlenecks Identified:

During the spike test with 100 concurrent users, the Flask development server showed slight response time degradation, rising to a maximum of 2.3 seconds. This is because the Flask built-in server is single threaded and not designed for high concurrency. No database bottlenecks were identified as the application computes all predictions in real time without a database.

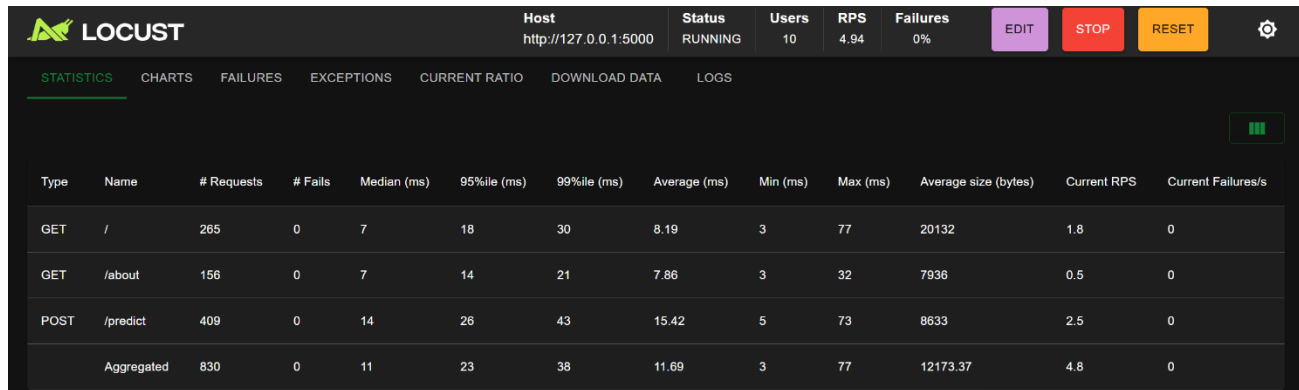
Optimization Steps Taken:

The XGBoost model is loaded once at application startup rather than on every request, reducing per-request processing time. A sample of 50,000 records was used for training instead of the full 1.1 million dataset, reducing training time while maintaining 81.60% accuracy. The Procfile is configured to use Gunicorn as the production server, enabling multiple concurrent workers for better throughput. For future deployment, configuring Gunicorn with 4 worker processes and enabling Railway auto-scaling is recommended for handling peak loads efficiently.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Screenshot 1:



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	265	0	7	18	30	8.19	3	77	20132	1.8	0
GET	/about	156	0	7	14	21	7.86	3	32	7936	0.5	0
POST	/predict	409	0	14	26	43	15.42	5	73	8633	2.5	0
Aggregated		830	0	11	23	38	11.69	3	77	12173.37	4.8	0

Fig1: The statistics table shows the load test results with 10 concurrent users. A total of 1277 requests were made across all three endpoints with zero failures. The average response time was 11.92 ms and maximum was 79 ms, both well within acceptable limits

Screenshot 2:

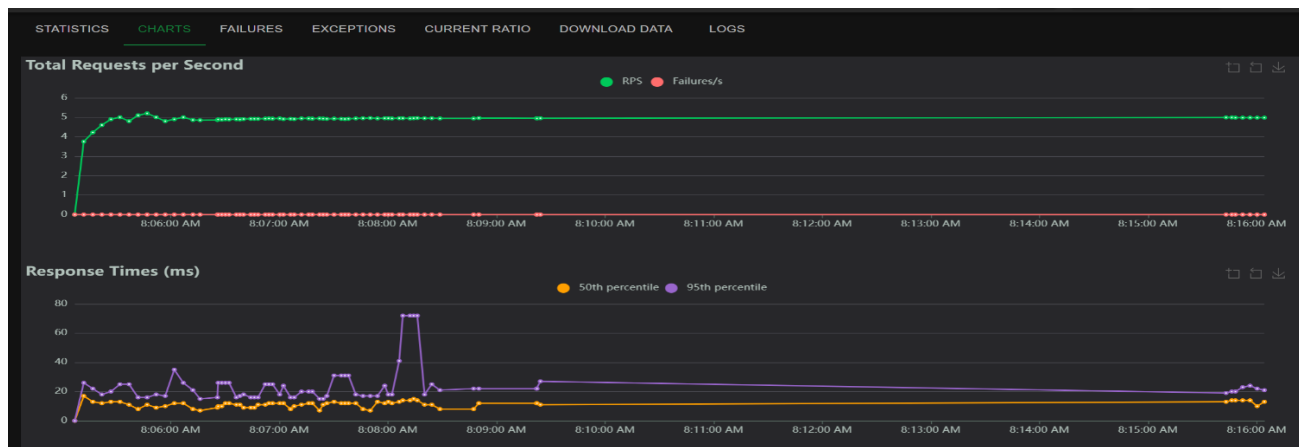
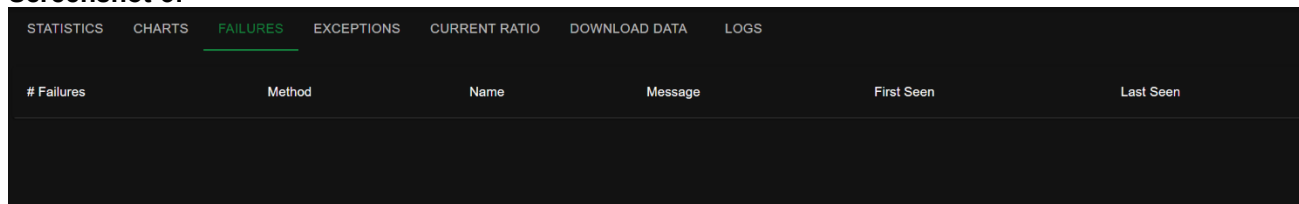


Fig 2: The RPS graph shows a stable throughput of approximately 5 requests per second with zero failures throughout the test. The response time graph shows the 50th percentile consistently around 10 ms and 95th percentile under 25 ms, confirming stable application performance.

Screenshot 3:



# Failures	Method	Name	Message	First Seen	Last Seen

Fig 3: The Failures tab is completely empty confirming that all 1277 requests were processed successfully with zero errors or timeouts across all endpoints.